

End of Course Project Report

Problem and proposed system

Online shoppers need clear answers about product facts and store terms. A fluent language model may use stale knowledge or confuse one model with another. I built an evidence-grounded Apple Product Support Assistant for shoppers in Singapore. It retrieves short notes paraphrased from Apple pages and asks a language model to answer from those notes. If the notes do not support a claim, it should say so. The prototype does not access orders, process returns or make warranty decisions.

The corpus has ten source-attributed notes on product specifications and Singapore store guidance. Each note links to an official source and a check date. The runner retrieves whole notes rather than overlapping chunks. I built the retrieval, prompt, logging and evaluation harness. Embeddings run locally; answer generation uses OpenRouter.

Data quality correction

Reviewing the original classroom corpus showed that it could not support its evaluation claims. It incorrectly described 96GB as the M3 Max maximum, iPhone 15 Pro Max playback as 24 hours, Apple Watch Ultra 2 water resistance as 85 metres, Trade In deadlines and limits, and a universal \$750 signature threshold. It also included unsupported return and promotion details. I withdrew the old 7/10 and 9/10 scores because they rewarded answers to unreliable facts. The original notebook and output remain in audit/ for review only; the runner uses the corrected corpus.

Official sources give the corrected facts: M3 Max supports up to 128GB (96GB is a 14-core configuration); iPhone 15 Pro Max supports up to 29 hours of video playback and 25 hours streamed; and Watch Ultra 2 supports EN13319 recreational diving to 40 metres, not below it. Apple Singapore return terms depend on eligibility and product-specific conditions. sources.md records the links and check dates. This correction changed both the system's evidence and the basis for scoring.

Evaluation design and results

The revised test set contains ten cases: five typical, three edge and two unsupported probes. It covers memory, battery, dive depth, returns, Trade In, compatibility, education-store limits, signatures, warranty and AppleCare+ price. A retrieval hit means the required note is present. A strict answer pass requires the necessary facts and qualifiers, with no forbidden claim. Refusal on unsupported cases is scored separately. The runner logs IDs, text, latency and token counts; answer meaning is judged manually. A literal L1 checker flags exact missing terms and forbidden phrases, but does not score paraphrases.

After correcting the corpus, I strengthened the prompt to state safety limits, supported corrections and order-specific next steps. The revised run retrieved the required note in all ten cases at both settings. Strict manual checks passed 7/10 at k=1 and 8/10 at k=2. Correct refusals passed 3/3 at both settings. Both Trade In answers omitted the required qualifier that device condition must match its description, so that case fails strict completeness at both settings. The k=1 Watch answer also missed the warning not to dive below 40m; k=2 included it. Both answers to the invented five-year warranty correctly refused the claim but omitted the supported one-year limited-warranty correction, so refusal passed while strict completeness failed. Those two answers used 11 completion tokens each and ended with a complete Evidence reference; there is no visible sign of truncation, but the run did not save the provider finish reason. Row-level judgments are in evaluation_results.csv; raw answers are in results/top_k_comparison_final_prompt_run.csv.

Metric	Target	k=1	k=2
Strict answer completeness	≥80%	7/10 (70%)	8/10 (80%)
Correct refusals	100%	3/3 (100%)	3/3 (100%)
Retrieval hit rate	Report	10/10	10/10
Median generation latency	Compare	1,261 ms	1,157 ms
Avg prompt/completion tokens	Compare	339/36	456/39

K=2 gained one net strict pass, including the missing dive-safety warning, while using about 34% more prompt tokens. Its median API-call latency was 104ms lower in this run, but one run cannot show that it is generally faster. The $\geq 80\%$ target was missed at k=1 and met at k=2. Latency excludes retrieval and the full user interaction.

The configured default model is qwen/qwen-2.5-7b-instruct, but the historical run did not record the effective model or any PE6201_MODEL override. At the listed OpenRouter rate (model pricing), the 7,947 input and 748 output tokens imply about \$0.00094 if that default was used. This is an estimate, not a verified charge; see results/run_metadata.json. Actual billing was not captured.

Business and technical trade-offs, risks and next steps

A keyword FAQ is inexpensive and predictable, but shoppers may need to search several pages and reconcile model names or regional terms. Semantic retrieval with a language model can combine source-backed facts into a short answer, but adds latency, token cost and the risk of unsupported claims. Local embeddings avoid sending retrieval text to an embedding API. K=1 uses less context and is easier to inspect; k=2 can retrieve a useful second note but costs more and may add distracting information. This corpus has one designated source per case, so it does not strongly test multi-source retrieval. I report retrieval separately from answer correctness so a fluent answer cannot hide a retrieval miss.

I checked all 20 Evidence references. Each ID appears in its retrieved context and points to the expected case source, but none uses the requested bracketed Evidence line on its own final line. This is a clear output-format defect. The automated check confirms ID alignment, not that every cited note supports every sentence. The source-level entailment still needs human review; details are in results/evidence_audit.csv.

The evaluation has ten self-authored questions, one small corpus, one run per setting and one human scorer. I tuned the prompt after seeing these same cases, so it may overfit this test set. There is no independent evaluator, statistical uncertainty estimate, multilingual test or currentness benchmark. Official pages can change, especially commercial terms, so the notes need regular updates. The prototype should direct users to current Apple sources for order-specific decisions.

A stronger next version would use independently written questions, more unsupported-policy probes, a second scorer and citation-entailment checks. It should also refresh sources regularly and test whether answers remain correct after policy changes. These steps would improve confidence, but a larger evaluation would still be needed before real customer use.

Conclusion

The corrected prototype retrieved every required note in this ten-case test, missed the 80% strict-answer target at k=1 and met it at k=2. K=2 reached 80% strict completeness with higher prompt-token use. The Trade In qualifier, warranty correction and Evidence format remain rough edges. The results support a limited prototype claim, not production reliability; the saved data and judgments make the outcome inspectable and open to improvement.